

# Chapter 8: Dataset

Gregory S. DeLozier, PhD

The chapters so far asked how much structure the programming layer should put on a database. The object-relational-mappers topic answered “a lot”: define models, and the models define the tables. This chapter tries the opposite answer. The `dataset` library asks for almost no structure. You hand it Python dictionaries, and it stores them in an ordinary relational database and gives dictionaries back.

That makes it a poor fit for an application that must guard its data, and a good fit for a different kind of work: opening a database you did not design, looking around, asking questions, keeping some intermediate results, and moving on. That work is called exploratory data analysis, and it is what analysts do before they know what the data contains. (1) The same habits apply when you build a dashboard or a report from a database that other people maintain.

We will use `dataset` three ways in this chapter: on the pets database we have been using, to rebuild the pets application with fewer lines of code and more attention to what the database enforces, and on an unfamiliar database called `mystery.db`, which we will explore without changing it.

Use the files in `chapter-08-dataset/code`.

## Installing Dataset

`dataset` is an ordinary Python package. Version 2.0.0 was released in April 2026 and needs Python 3.10 or newer. (2) It is built on SQLAlchemy, which supplies the database connections and SQL generation, and on Alembic, which it uses to change table definitions. (3)(4) Installing `dataset` installs both.

Work in a virtual environment so the packages stay with this project. (5)

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

On Windows, the second command is `.venv\Scripts\activate`. The `requirements.txt` in the code folder pins the versions this chapter was tested with:

```
dataset==2.0.0
Flask==3.1.3
tzdata==2026.4
```

Check the installation:

```
python3 -m pip show dataset
```

You should see `Name: dataset` and `Version: 2.0.0`. `dataset` supports SQLite, PostgreSQL, and MySQL. We will use SQLite, because it needs no server. (6)

## Looking at a Database You Did Not Design

The code folder includes a `pets.db` made from `create_db.sql`, and it may have been changed by earlier use. To match the output below, rebuild it. The script refuses to overwrite a file, so remove the old one first, along with the two extra files `dataset` creates, which the section on what `dataset` leaves to you explains:

```
rm -f pets.db pets.db-shm pets.db-wal
bash create_db.sh
```

Then open it from Python:

```
import dataset

db = dataset.connect("sqlite:///pets.db")
print(db.tables)
print(db["pets"].columns)
print(len(db["pets"]))
```

```
['kind', 'pets']
['id', 'name', 'age', 'kind_id', 'owner']
4
```

The connection string names the kind of database and where it is. `db.tables` lists the tables, `db["pets"]` gives a table object, `.columns` lists its column names, and `len()` counts its rows. No model classes were written. `dataset` read the definitions from the database itself. (6)

Rows come back as dictionaries:

```
suzy = db["pets"].find_one(name="Suzy")
print(suzy)
print(suzy["age"])
```

```
OrderedDict({'id': 1, 'name': 'Suzy', 'age': 3, 'kind_id': 1, 'owner': 'Greg'})
3
```

The row is an ordered dictionary, a kind of `dict` that remembers column order, so everything you know about dictionaries applies: `row["age"]`, `row.get("age")`, `dict(row)`, and JSON conversion.

The same code works on a database made by different software. The database from the `orm-with-constraints` topic was created by `Peewee`, and `dataset` can read it. Open it read-only, so that looking cannot change anything:

```
db7 = dataset.connect("sqlite:///file:pets.db?mode=ro&uri=true", sqlite_wal_mode=False)
print(db7.tables)
for name in db7.tables:
    print(name, db7[name].columns, len(db7[name]))
```

```
['owner', 'pet']
owner ['id', 'name', 'city', 'type_of_home'] 2
pet ['id', 'name', 'type', 'age', 'owner_id'] 5
```

Run this from the chapter 7 code folder, or copy that folder's `pets.db` somewhere first. The `mode=ro` part of the connection string is an SQLite feature that refuses every write. (7) The `sqlite_wal_mode=False` argument matters too, and the section on what dataset leaves to you explains why.

## Asking Questions

Filtering is done with keyword arguments. Each keyword names a column, and a plain value means “equals”:

```
pets = db["pets"]
pets.find_one(name="Suzy")
pets.find(kind_id=1)
pets.find(kind_id=[1, 3])
pets.find(age={">=": 3}, order_by="-age", _limit=2)
```

A list means “any of these”. A dictionary applies an operator: `>`, `>=`, `<`, `<=`, `!=`, `in`, `notin`, `like`, `ilike`, `between`, `startswith`, and `endswith` are available. (8) `order_by` takes a column name, and a leading minus sign means descending. `_limit` and `_offset` page through results. `find` returns a result you can loop over. `find_one` returns one row, or `None` when nothing matches.

```
for pet in pets.find(age={">=": 2}, order_by="-age"):
    print(pet["name"], pet["age"])
```

```
Heidi 4
Suzy 3
Sandy 2
```

Other questions have their own methods. `pets.count(kind_id=1)` counts matching rows. `pets.distinct("owner")` lists the distinct values of a column. `len(pets)` counts them all.

When the convenience methods run out, write SQL. `db.query` runs a statement and returns rows as dictionaries:

```
for row in db.query("select kind_id, count(*) as n from pets group by kind_id"):
    print(dict(row))
```

```
{ 'kind_id': 1, 'n': 2 }
{ 'kind_id': 2, 'n': 1 }
{ 'kind_id': 3, 'n': 1 }
```

dataset does not hide SQL from you. It postpones it. Joins, grouping, and anything else that needs SQL still take SQL, and the results still arrive as dictionaries. (6)

## Changing Data

The write methods take dictionaries too:

```
pets.insert({"name": "Buddy", "age": 5, "owner": "Alice", "kind_id": 1})
pets.insert_many([
    {"name": "Rex", "age": 2, "owner": "Sam", "kind_id": 1},
    {"name": "Tom", "age": 6, "owner": "Sam", "kind_id": 2},
])
pets.update({"name": "Buddy", "age": 6}, ["name"])
pets.upsert({"name": "Buddy", "age": 7}, ["name"])
pets.delete(name="Tom")
```

`insert` returns the new row's identifier. `update` takes a dictionary of values and a list of the columns that identify the row, and it returns the number of rows changed. `upsert` updates the matching row if there is one, and returns `True`; if there is none, it inserts a new row and returns the new row's identifier. `delete` takes the same keyword filters as `find` and returns `True` if it removed anything. Called with no filters, `delete` removes every row in the table.

`insert_many` works in chunks, which makes it the right tool for loading a large file. `upsert_many` exists, but in version 2.0.0 it simply calls `upsert` once per row, so it is slow for a large file and its `chunk_size` argument has no effect. (8)

Related changes belong together. A transaction is a group of changes that succeed or fail as a unit. A connection used with `with` is a transaction, and an exception inside the block undoes everything in it:

```
with dataset.connect("sqlite:///pets.db") as tx:
    tx["pets"].insert({"name": "Inside", "age": 1, "owner": "Sam", "kind_id": 1})
    tx["pets"].insert({"name": "Bad", "age": -1, "owner": "Sam", "kind_id": 1})
```

The second insert breaks a rule in the table, the error leaves the block, and the first pet is not stored either.

## Tables That Make Themselves

Create a table by using it:

```
scratch = db["scratch"]
scratch.insert({"a": 1, "b": 2.5, "c": True, "d": "hi"})
```

`dataset` creates the table with an integer `id` that counts up by itself, and one column for each key in the dictionary. It chooses each column's type from the Python value it saw. An integer becomes a `BIGINT`, a float becomes a `FLOAT`, a boolean becomes a `BOOLEAN`, a date or `datetime` becomes a date or timestamp column, a dictionary becomes a `JSON` (JavaScript Object Notation) column, and everything else becomes `TEXT`. (6)

The same rule applies to columns. Insert a dictionary with a key the table has never seen, and `dataset` adds the column:

```
print(pets.columns)
pets.insert({"name": "Lucky", "age": 3, "owner": "Ana", "kind_id": 1, "breed": "Beagle"})
print(pets.columns)
print(pets.find_one(name="Suzy"))
```

```
['id', 'name', 'age', 'kind_id', 'owner']
['id', 'name', 'age', 'kind_id', 'owner', 'breed']
OrderedDict({'id': 1, 'name': 'Suzy', 'age': 3, 'kind_id': 1, 'owner': 'Greg', 'breed': None})
```

The older rows now have a `breed` column, and its value for them is `NULL`. For analysis this is convenient. You can keep a scratch table for intermediate results, add a column when a new idea needs one, and never write a `CREATE TABLE` statement.

It is also how a typo becomes part of your schema. Insert `{"name": "Heidi", "aeg": 15}` and `dataset` adds an `aeg` column without comment. Nothing complains, and the mistake is now in the table definition, waiting for the next person who runs a query. The remedy is to turn the behavior off when you are working with a database that matters. Passing `ensure_schema=False` to `dataset.connect` stops it from creating tables and columns. A table that does not exist then raises an error, and a key that is not a column is dropped from the insert without a message. That is safer for the schema, but it means a misspelled key loses that value quietly, so check what you wrote.

## What Alembic Does

When `dataset` adds the `breed` column, it does not build the SQL itself. It asks Alembic. Alembic is a database migration tool that works with SQLAlchemy. (4) A **migration** is a change to a database’s structure, made in a way that can be repeated and tracked. (9)

Alembic is normally used as a discipline for changing a database over time. A project keeps a folder of migration scripts, each one a numbered step such as “add a `breed` column to `pets`”. Each script has an upgrade direction and a downgrade direction. Alembic records in the database, in a table named `alembic_version`, which step it has reached, so that every copy of the database can be brought to the same version, in order. (4)

`dataset` uses only a small part of that. Alembic has an operations layer that can add a column to a table by producing the right `ALTER TABLE` statement for whichever database is in use. `dataset` calls it whenever it sees a new key. On SQLite the statement is `ALTER TABLE ... ADD COLUMN`, which SQLite supports directly. (10) The migration scripts, version numbers, and downgrades are not used. After the insert above, the database has no `alembic_version` table. It has the new column and nothing that records how it got there.

That difference is the reason to be careful with the word “migration” here. Earlier chapters kept database changes in a script that a person reads and runs. A real Alembic setup keeps them in versioned files that the team reviews. With `dataset`, the changes happen as a side effect of an insert, and no one reviews them. That is the right trade for scratch work and the wrong one for a database that other programs depend on.

## What Dataset Leaves to You

**Rules written in SQL still apply.** The pets database was created from `create_db.sql`, which declares `NOT NULL` on the name and `CHECK (age >= 0)` on the age. `dataset` did not create those rules and does not check them, but SQLite enforces them. Inserting a pet with no name or an age of `-5` raises an `IntegrityError` that names the failed constraint. This is the same protection the database-constraints topic built.

**Rules that were never written are not there.** A table that `dataset` created from dictionaries gets very little beyond a primary key and guessed column types. Application rules are not inferred. Inserting `{"name": "Bad", "age": "not a number"}` into a table whose age column has no `CHECK` stores the text, because SQLite is willing to keep text in an integer column. (6)

**Foreign keys are off.** SQLite ignores foreign keys unless each connection turns them on. (11) Without that, `dataset` will insert a pet whose `kind_id` names no kind, and it will delete a kind that pets still use. You can turn the setting on when you connect:

```
db = dataset.connect("sqlite:///pets.db", on_connect_statements=["PRAGMA foreign_keys=ON"])
```

The statements in `on_connect_statements` run on every new connection. With foreign keys on, both mistakes raise an `IntegrityError`.

**Relationships are yours to manage.** `dataset` has no idea that `pets.kind_id` refers to `kind.id`. The earlier version of the pets application looked up each pet's kind with a separate `find_one`, so a list of one hundred pets ran one hundred and one queries. That is the N+1 problem from the orm-with-constraints topic, and it appears here for the same reason. A join in `db.query` runs one statement and does the whole job.

**It changes the file when it connects.** By default, connecting to an SQLite file makes `dataset` switch the database to write-ahead logging, which is where the `pets.db-wal` and `pets.db-shm` files come from. In this mode SQLite records changes in a separate log file and shares some state through the shared-memory file. (12) It is a sensible choice for a program that writes, and a surprise when all you meant to do was look. Passing `sqlite_wal_mode=False` turns it off for that connection. Combined with a read-only connection, it means exploring a file leaves the file exactly as it was.

There is one catch. The logging mode is stored in the database file. A file that any earlier `dataset` connection has opened with the default settings stays in that mode, and SQLite may create the two extra files again whenever the file is opened, even read-only. To put such a file back, run `sqlite3 pets.db "pragma journal_mode=delete"`. For a file you did not make, keep a copy and explore the copy.

## The Pets Application with Dataset

The pets application in `app.py` has the same routes as before, and it manages the same table of pets and table of kinds. Most of its code is dictionary handling. Here is the route that lists the pets:

```
@app.route("/")
@app.route("/list")
def get_list():
    pets = list(db.query(
        "select pets.id, pets.name, pets.age, pets.owner, "
        "kind.kind_name, kind.food, kind.noise "
        "from pets join kind on kind.id = pets.kind_id "
        "order by pets.id"
    ))
    return render_template("list.html", pets=pets)
```

One query replaces the loop of lookups. The connection at the top of the file turns foreign keys on, and it reads the database location from an environment variable so that the tests can use a temporary file:

```
DATABASE_URL = os.environ.get("PETS_DATABASE_URL", "sqlite:///pets.db")
db = dataset.connect(DATABASE_URL, on_connect_statements=["PRAGMA foreign_keys=ON"])
```

The create route checks its form before it writes anything, and it catches the database's refusal if a bad value gets past the check:

```
problem = check_pet_form(request.form)
if problem:
    return error_page(problem)
try:
    db["pets"].insert(pet_values(request.form))
except IntegrityError as error:
    return error_page(f"Constraint error creating pet: {error.orig}")
```

These are the same layers as in the `orm-with-constraints` topic. The form marks its fields required and the age field `type="number" min="0"`, so the browser answers first. `check_pet_form` checks the request. The table's NOT NULL and CHECK rules and the foreign key are the last line. Deleting a kind that pets use no longer needs a hand-written check in the route. The database refuses, and the route turns the refusal into a message.

Two things are different from the Peewee version. There is no `database.py` between the routes and the library, because `dataset` calls are short enough to use directly. And the rules live in `create_db.sql`, not in Python, because `dataset` has no model layer in which to declare this application's CHECK and foreign key rules. It can add a column with `nullable=False` or `unique=True` and it can create an index, but those calls are one-off schema changes, not a description of the whole design. If the SQL script does not contain a rule, no layer below the route has it.

Run the application and its tests:

```
python3 -m unittest -v test_app.py
flask --app app run --debug
```

Open <http://127.0.0.1:5000/list>.

## Exploring a Mystery Database

Now the situation dataset was made for. Someone gives you a database file and asks what is in it. You do not have a schema diagram, and you are not going to change anything.

The script `build_mystery.py` creates such a file. It uses the Python standard library's `unicodedata` module and `zoneinfo` module. On systems without an operating system time zone database, such as Windows, the `tzdata` package (listed in `requirements.txt`) supplies one. (13)(14) Run it once, and then avoid reading it until you have explored the result:

```
python3 build_mystery.py
```

```
Built mystery.db (4340 KB, Unicode 16.0.0).
```

Every count in this exercise depends on the Unicode version built into your copy of Python and on the time zone data installed. The printed numbers came from Python 3.14 with Unicode 16.0.0. Python 3.10 uses Unicode 13.0 and builds 57,224 character rows and 1,552 scripts. Python 3.11 uses Unicode 14.0 and builds 57,390 and 1,570. So your counts will differ a little. The lessons do not change; check your own output rather than matching these numbers.

The script will not overwrite an existing file unless you add `--replace`. The script `explore_mystery.py` collects the steps below. Open the database read-only:

```
db = dataset.connect("sqlite:///file:mystery.db?mode=ro&uri=true", sqlite_wal_mode=False)
```

**Step 1: What is here?** List the tables and views. A view is a saved query that behaves like a table.

```
print(db.tables)
print(db.views)
```

```
['category', 'character', 'timezone']
['numeric_character', 'script_summary']
```

**Step 2: What is in each one?** Loop over them and print the columns and row counts.

```
category: 30 rows
  columns: ['code', 'description', 'major_class']

character: 57462 rows
  columns: ['codepoint', 'symbol', 'name', 'category', 'script', 'width', 'numeric_value']

timezone: 518 rows
  columns: ['name', 'region', 'city', 'january_offset_minutes', 'july_offset_minutes',
'uses_dst']

numeric_character: 810 rows
  columns: ['codepoint', 'symbol', 'name', 'numeric_value']

script_summary: 1601 rows
  columns: ['script', 'characters', 'first_codepoint', 'last_codepoint']
```

Names carry a lot of meaning. `character` looks like it describes text characters, `timezone` is unrelated to it, and `category` looks like a lookup table for the `category` column in `character`. Those are guesses. We can test them.

**Step 3: Look at some rows.**

```
for row in db["character"].find(_limit=3, order_by="codepoint"):
    print(dict(row))
```

```
{'codepoint': 32, 'symbol': ' ', 'name': 'SPACE', 'category': 'Zs', 'script': 'SPACE',
'width': 'Na', 'numeric_value': None}
{'codepoint': 33, 'symbol': '!', 'name': 'EXCLAMATION MARK', 'category': 'Po', 'script':
'EXCLAMATION', 'width': 'Na', 'numeric_value': None}
{'codepoint': 34, 'symbol': '"', 'name': 'QUOTATION MARK', 'category': 'Po', 'script':
'QUOTATION', 'width': 'Na', 'numeric_value': None}
```

Each row is one character. The category codes look like the codes in the small `category` table. The `script` column is more puzzling: a space is not a script. Keep that in mind.

**Step 4: Test the guesses with a question that uses two tables.**

```
rows = db.query(
    "select category.description, count(*) as characters "
    "from character join category on category.code = character.category "
    "group by category.code order by characters desc limit 5")
for row in rows:
    print(dict(row))
```

```
{'description': 'Other letter', 'characters': 46126}
{'description': 'Other symbol', 'characters': 4607}
{'description': 'Lowercase letter', 'characters': 1448}
{'description': 'Uppercase letter', 'characters': 1132}
{'description': 'Nonspacing mark', 'characters': 1066}
```

The join worked, so the guess about category was right. Most characters are “Other letter”, which suggests the data includes large writing systems whose letters are not upper or lower case.

**Step 5: Use the views.** Someone already wrote queries worth reading. The `script_summary` view groups by the `script` column.

```
for row in db["script_summary"].find(order_by="-characters", _limit=5):
    print(dict(row))
```

```
{'script': 'CJK', 'characters': 28209, 'first_codepoint': 11904, 'last_codepoint': 64217}
{'script': 'HANGUL', 'characters': 11625, 'first_codepoint': 4352, 'last_codepoint': 55291}
{'script': 'YI', 'characters': 1220, 'first_codepoint': 40960, 'last_codepoint': 42182}
{'script': 'ARABIC', 'characters': 1185, 'first_codepoint': 1536, 'last_codepoint': 65276}
{'script': 'LATIN', 'characters': 1183, 'first_codepoint': 65, 'last_codepoint': 64262}
```

The view has 1,601 rows, and real writing systems number far fewer than that. Together with the `SPACE` row from step 3, this says the `script` column is the first word of each character’s name, and not a real classification. The largest groups happen to be real scripts because their names begin with the script name. Finding that out is a result. An analysis that grouped by `script` and reported the answer without looking at the values would be wrong.

**Step 6: Find the interesting values.** The `numeric_character` view holds characters that stand for numbers.

```
for row in db["numeric_character"].find(order_by="-numeric_value", _limit=3):
    print(row["codepoint"], row["name"], row["numeric_value"])
for row in db["numeric_character"].find(numeric_value=7, _limit=5):
    print(row["name"])
```

```
20140 CJK UNIFIED IDEOGRAPH-4EAC 1e+16
31213 CJK UNIFIED IDEOGRAPH-79ED 1000000000.0
20159 CJK UNIFIED IDEOGRAPH-4EBF 100000000.0
DIGIT SEVEN
ARABIC-INDIC DIGIT SEVEN
EXTENDED ARABIC-INDIC DIGIT SEVEN
NKO DIGIT SEVEN
DEVANAGARI DIGIT SEVEN
```

The largest values are Chinese numerals. The 7 query finds the digit seven from several writing systems.

**Step 7: Look at the unrelated table.** The `timezone` table holds each zone’s offset from UTC in January and July, in minutes. UTC is Coordinated Universal Time, the reference that offsets are measured from.

```
rows = db.query(
    "select name, january_offset_minutes from timezone "
    "where january_offset_minutes % 60 != 0 order by january_offset_minutes limit 5")
for row in rows:
    print(dict(row))
```

```
{'name': 'Pacific/Marquesas', 'january_offset_minutes': -570}
{'name': 'America/St_Johns', 'january_offset_minutes': -210}
{'name': 'Canada/Newfoundland', 'january_offset_minutes': -210}
{'name': 'Asia/Tehran', 'january_offset_minutes': 210}
{'name': 'Asia/Kabul', 'january_offset_minutes': 270}
```

These zones sit a half hour or forty-five minutes away from a whole-hour offset. America/St\_Johns and Canada/Newfoundland are one place under two names, because the time zone database keeps older names as aliases. Notice that before you count zones.

**Step 8: Keep what you found.** Results usually leave the exploration. A CSV file (comma-separated values) is the common exchange format, and Python's csv module writes one from the dictionaries:

```
import csv

rows = list(db["script_summary"].find(order_by="-characters"))
with open("scripts.csv", "w", newline="") as output:
    writer = csv.DictWriter(output, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)
```

Earlier versions of dataset had an export feature. Since version 1.0 it lives in a separate package, datafreeze, so the standard library is the simpler choice. (15)

Because the connection was read-only, the exploration made no change to `mystery.db`. If you need a scratch table for intermediate results, create a second database file for it. dataset makes that a two-line job, and the original stays untouched.

## Choosing a Tool

|                       | Direct SQL          | Peewee                      | Dataset                                  |
|-----------------------|---------------------|-----------------------------|------------------------------------------|
| You write             | SQL statements      | Model classes               | Dictionaries and filters                 |
| The schema comes from | SQL scripts         | The models                  | The database, or the first rows inserted |
| Rules are declared in | SQL                 | The models                  | SQL, outside the library                 |
| Relationships         | Joins               | Foreign key fields          | Joins in SQL                             |
| Changing a table      | ALTER TABLE by hand | Manual, or a migration tool | Automatic, unreviewed                    |
| Suits                 | Full control        | Applications with rules     | Exploration, scripts, scratch work       |

Choose dataset when you are looking at data, loading it, or reshaping it, and when a wrong guess costs you little. Choose Peewee, or hand-written SQL behind a small database module, when the code will serve other people and the data must follow rules.

## Practice

Work in a copy of the code directory.

1. Create the virtual environment, install the requirements, and run `python3 -m unittest -v`. Read the test names, and find the ones about automatic columns and the read-only connection.
2. Run `explore_mystery.py`. Then answer a question of your own with `db.query`: for example, which time zones have the largest difference between their January and July offsets?
3. In an interactive session, insert a pet with a misspelled key, then print the columns. Rebuild `pets.db`, connect with `ensure_schema=False`, and repeat the insert. What happens to the misspelled value?
4. Connect to `pets.db` without `on_connect_statements` and delete a kind that pets use. Then repeat with the foreign key setting on. Rebuild the database between attempts, as shown at the start of the chapter.
5. Rebuild `pets.db`, run `ls`, connect with the default settings, and run `ls` again. Then rebuild the file, connect with `sqlite_wal_mode=False`, and compare. Which extra files appear, and when? Use `sqlite3 pets.db "pragma journal_mode"` to see the mode.
6. Open the `pets.db` from the `orm-with-constraints` topic read-only. Write one `db.query` that lists each owner and the number of pets that owner has.
7. Write a script that copies the `character` rows for one writing system into a new scratch database with `insert_many`, then explore the scratch database.

## Exercise: Explore Something Else

Choose an SQLite database you did not create: a sample database such as Chinook, or a file from another course.

(16) Explore it the way we explored `mystery.db`, in order: tables and views, columns and counts, a few rows, a join to test a guess, and a look at any column whose values seem odd. Write down each guess before you test it, and note which ones were wrong. Save one result to a CSV file. Then say which step you would have to repeat if the database were much larger, and what you would add to make it faster.

## References

All sources accessed September 24, 2026.

1. Wikipedia contributors. “Exploratory data analysis.” Wikipedia. [https://en.wikipedia.org/wiki/Exploratory\\_data\\_analysis](https://en.wikipedia.org/wiki/Exploratory_data_analysis)
2. Friedrich Lindenberg, Gregor Aisch, and Stefan Wehrmeyer. “dataset.” Python Package Index. <https://pypi.org/project/dataset/>
3. SQLAlchemy. “SQLAlchemy 2.0 Documentation.” <https://docs.sqlalchemy.org/en/20/>
4. Alembic. “Alembic Documentation.” <https://alembic.sqlalchemy.org/en/latest/>
5. Python Software Foundation. “venv: Creation of virtual environments.” Python documentation. <https://docs.python.org/3/library/venv.html>
6. Friedrich Lindenberg, Gregor Aisch, and Stefan Wehrmeyer. “Quickstart.” dataset documentation. <https://dataset.readthedocs.io/en/latest/quickstart.html>
7. SQLite. “URI Filenames in SQLite.” <https://www.sqlite.org/uri.html>

8. Friedrich Lindenberg, Gregor Aisch, and Stefan Wehrmeyer. “API documentation.” dataset documentation. <https://dataset.readthedocs.io/en/latest/api.html>
9. Wikipedia contributors. “Schema migration.” Wikipedia. [https://en.wikipedia.org/wiki/Schema\\_migration](https://en.wikipedia.org/wiki/Schema_migration)
10. SQLite. “ALTER TABLE.” [https://www.sqlite.org/lang\\_altertable.html](https://www.sqlite.org/lang_altertable.html)
11. SQLite. “SQLite Foreign Key Support.” <https://www.sqlite.org/foreignkeys.html>
12. SQLite. “Write-Ahead Logging.” <https://www.sqlite.org/wal.html>
13. Python Software Foundation. “unicodedata: Unicode Database.” Python documentation. <https://docs.python.org/3/library/unicodedata.html>
14. Python Software Foundation. “zoneinfo: IANA time zone support.” Python documentation. <https://docs.python.org/3/library/zoneinfo.html>
15. Friedrich Lindenberg. “dataset.” GitHub. <https://github.com/pudo/dataset>
16. Luis Rocha. “Chinook Database.” GitHub. <https://github.com/lerocha/chinook-database>

## Questions for Thought

1. `dataset` adds a column when it sees a new key. When is that a feature, and when is it a hazard? What would you want to check before turning it on for a database other people use?
2. Alembic tracks versions and supports downgrades, and `dataset` uses none of that. What is lost when schema changes happen as a side effect of an insert?
3. The `pets` application relies on the database for its constraints. What did it cost to move the rules out of Python, and what did it gain?
4. In the `mystery` database, the `script` column looked useful and was not. What habits would have caught that if you had not looked at the values?
5. Why does connecting with the default settings count as changing a database, even if you never write a row? When does that matter?
6. Which parts of your own work look like exploration, and which look like building an application? Where would each tool from this chapter fit?